1. Motivation for the AI model
Especially with today's political climate, we noticed that gasoline prices fluctuate week to week and directly affect household budgets, commuting costs, and the operating costs of local businesses that depend on vehicles (delivery services, rideshare drivers, landscaping companies, etc.). This project builds a short-term forecasting model for the average weekly price of Regular gasoline in the Charlotte-Gastonia-Rock Hill metro area. The goal is to give consumers and small businesses a simple, data-driven way to anticipate whether prices are trending up or down over the coming weeks, so they can time purchases or budget more accurately instead of reacting to prices after the fact.

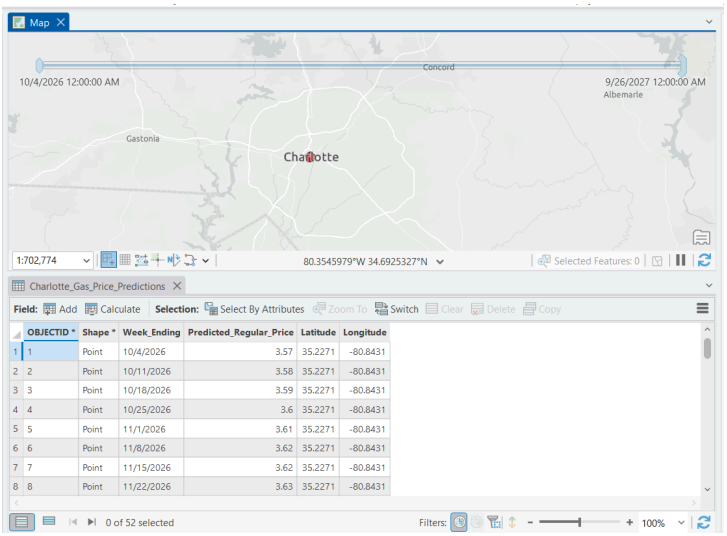
2. Why we chose to train your model on this data
The training data was compiled from historical AAA gas price averages for the Charlotte-Gastonia-Rock Hill metro, sourced from the `lykmapipo/US-Gas-Prices` GitHub repository — a scraper that commits a new dated CSV snapshot to its `data/metro-daily-averages/` folder each day it runs. Since older snapshots get overwritten in the working directory but remain in git history, the full historical series was reconstructed by walking the repo's commit log and pulling out every version of the file, then filtering each day's snapshot down to just the Charlotte-Gastonia-Rock Hill row. This gave 102 weekly data points (resampled from daily) spanning December 2023 through today. This dataset was chosen because:

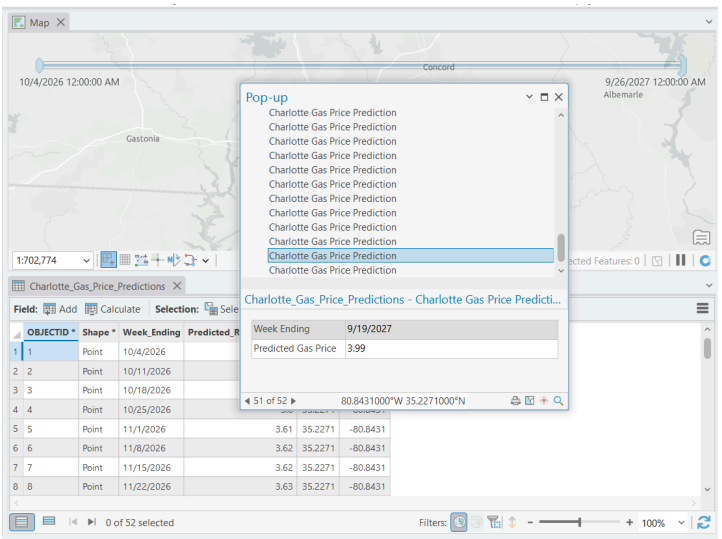
- It's the exact quantity we want to forecast, so no proxy variable is needed.
- It's a real, continuously updating public feed (AAA, via an active scraper) rather than a one-time static download, which matches the assignment's requirement that the pipeline eventually pull from a live source.
- Daily granularity (resampled to weekly) smooths short-term noise while preserving trend and seasonal movement.
- Reconstructing the series from git history also gave us meaningfully more historical depth than a single current snapshot would, which matters for training a stable trend line.
- The date field converts cleanly into a simple numeric feature (days since the start of the series), enabling a working linear regression baseline with minimal preprocessing.

3. Output of the model
We chose a **regression model** because it outputs a single continuous number: the predicted average price per gallon (in USD) of Regular gasoline in the Charlotte-Gastonia-Rock Hill metro for a given future week — conceptually the same as a weather app predicting a temperature value (e.g., 72°F) rather than a category like "hot" or "cold."

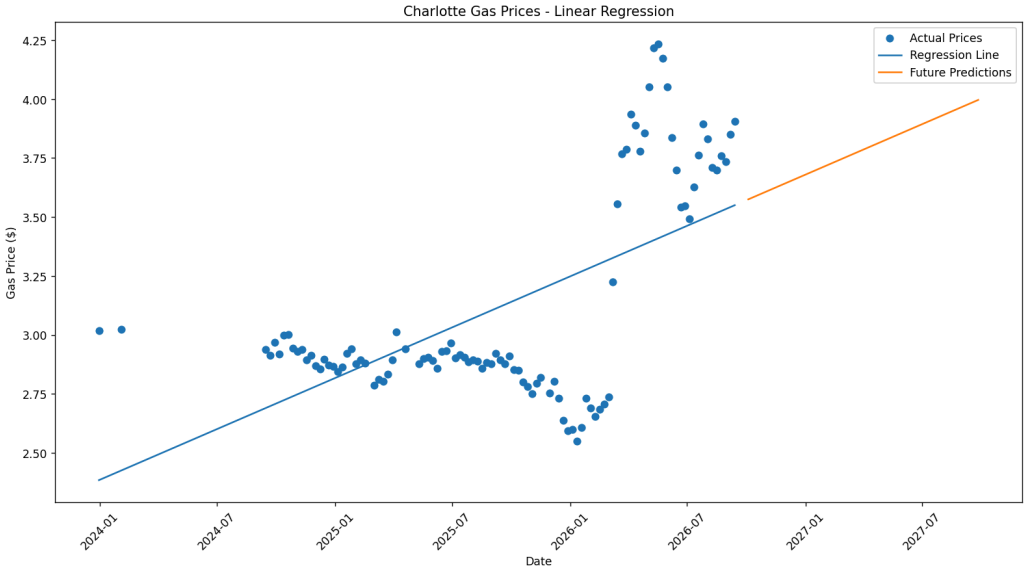
To make these results spatially intuitive and accessible, we integrated our model's 2027 out-of-sample forecasts into ArcGIS, building an interactive map that visualizes projected price variations across the region. Additionally, our code generated a linear regression trend and forecast graph comparing historical actuals against model predictions.

These are screenshots of the interactive map made in ArcGIS which will be demonstrated in our Zoom recording:





This is a screenshot of the linear regression model generated by our python code:



4. Steps completed in the demo

- Located a live, continuously-updating public gas price data source (AAA data via the `lykmapipo/US-Gas-Prices` scraper repo) and reconstructed its full historical coverage by extracting every dated snapshot from the repo's git commit history.
- Generated in-sample predictions and 52-week out-of-sample forecasts, and visualized actual vs. fitted vs. forecast prices on a single chart.
- Our data pipeline has two components. First, we use the GitHub REST API to retrieve the most recent daily gas price file from a public repository that archives AAA's metro-level gas price data; our Python code requests the folder listing, selects the newest CSV by date, downloads it, and filters it to the Charlotte metro area. Second, we use Amazon S3 as our cloud storage: the filtered API data is uploaded to our S3 bucket using the boto3 library, and our historical weekly training dataset is stored there as well. When the model runs, it reads the training data directly from S3, so the data is stored centrally in the cloud rather than on a single computer.
    - Pulled the newest data from GitHub's API and filtered it to Charlotte
    - Uploaded the Charlotte snapshot to an S3 bucket
    - Loaded the training data from S3
    - Trained a linear regression model on date vs. Regular price ($R^2 = 0.37$)
    - Generated 52 weeks of predictions
    - Built an interactive prediction where a typed date returns a price
    - Exported a CSV with coordinates for ArcGIS and made a graph

5. Steps remaining before the end of the semester

- Evaluate the linear regression model and explore any additional methods to better capture gas price fluctuations and seasonal trends.
- Continue collecting weekly AAA gas prices and retrain the model with updated data.
- Refine the ArcGIS interactive map and forecast graph to make predictions more comprehensible.
- Finalize and organize the project code, data, and documentation by October 27.
- Incorporate feedback from the November 3 code review.
- Complete the final documentation, demonstrate the model, discuss limitations, and present the project by the end of the semester.
- For the remainder of the model we will have it predict diesel and premium gas as well.